

Review

- What is security: history and definition
- Security policy, mechanisms and services
- Security models

Outline

- Overview of Cryptography
- Classical Symmetric Cipher
- Modern Symmetric Ciphers (DES)

Basic Terminology

- **plaintext** - the original message
- **ciphertext** - the coded message
- **cipher** - algorithm for transforming plaintext to ciphertext
- **key** - info used in cipher known only to sender/receiver
- **encipher (encrypt)** - converting plaintext to ciphertext
- **decipher (decrypt)** - recovering ciphertext from plaintext
- **cryptography** - study of encryption principles/methods
- **cryptanalysis (codebreaking)** - the study of principles/methods of deciphering ciphertext *without* knowing key
- **cryptology** - the field of both cryptography and cryptanalysis

Classification of Cryptography

- Number of keys used
 - Hash functions: no key
 - Secret key cryptography: one key
 - Public key cryptography: two keys - public, private
- Type of encryption operations used
 - substitution / transposition / product
- Way in which plaintext is processed
 - block / stream

Secret Key vs. Secret Algorithm

- Secret algorithm: additional hurdle
- Hard to keep secret if used widely:
 - Reverse engineering, social engineering
- Commercial: published
 - Wide review, trust
- Military: avoid giving enemy good ideas

Cryptanalysis Scheme

- Ciphertext only:
 - Exhaustive search until "recognizable plaintext"
 - Need enough ciphertext
- Known plaintext:
 - Secret may be revealed (by spy, time), thus <ciphertext, plaintext> pair is obtained
 - Great for monoalphabetic ciphers
- Chosen plaintext:
 - Choose text, get encrypted
 - Pick patterns to reveal the structure of the key

Unconditional vs. Computational Security

- **Unconditional security**

- No matter how much computer power is available, the cipher cannot be broken
- The ciphertext provides insufficient information to uniquely determine the corresponding plaintext
- Only one-time pad scheme qualifies

- **Computational security**

- The cost of breaking the cipher exceeds the value of the encrypted info
- The time required to break the cipher exceeds the useful lifetime of the info

Brute Force Search

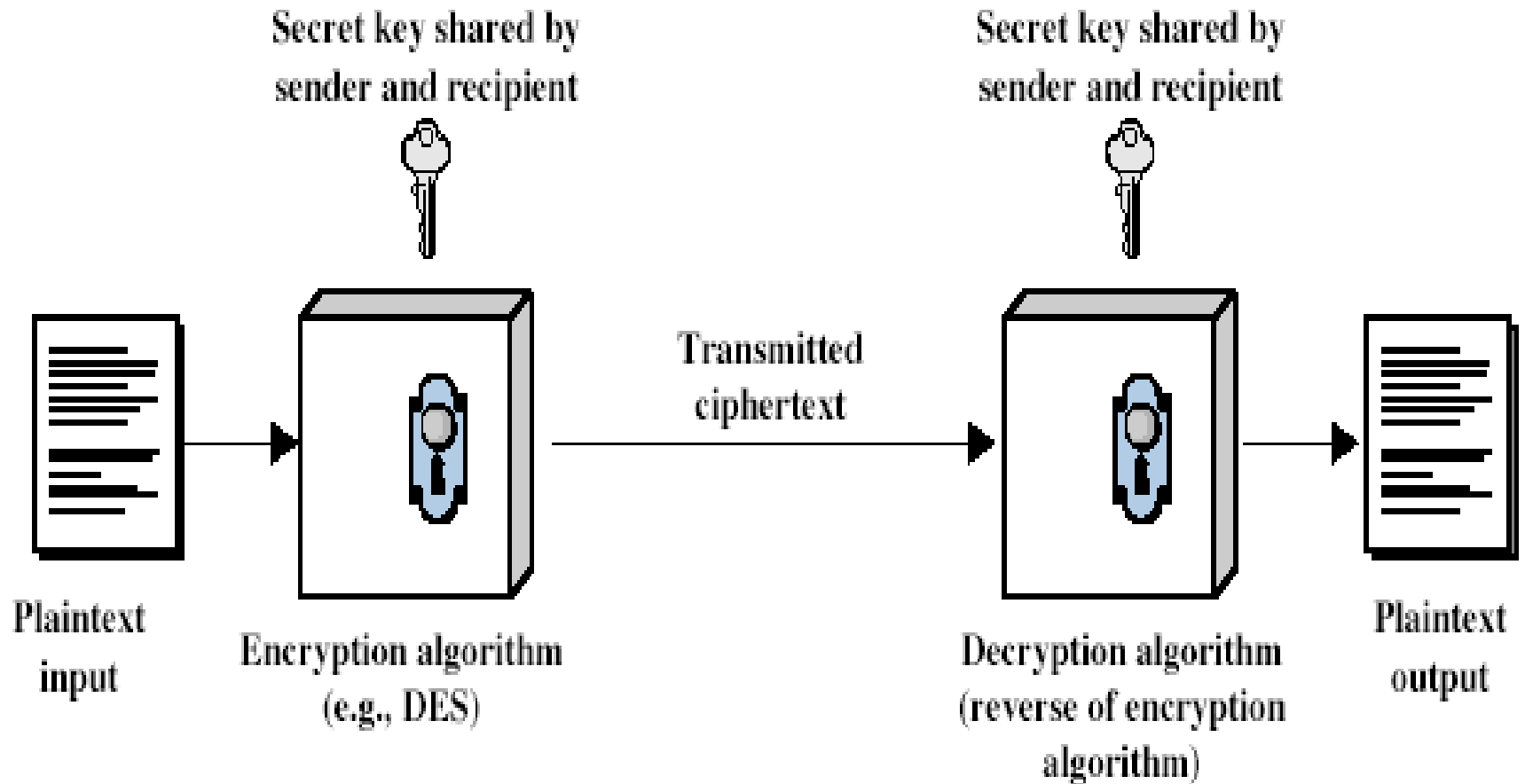
- Always possible to simply try every key
- Most basic attack, proportional to key size
- Assume either know / recognise plaintext

Key Size (bits)	Number of Alternative Keys	Time required at 1 decryption/ μ s	Time required at 10^6 decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24}$ years	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36}$ years	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12}$ years	$6.4 \times 10^6 \text{ years}$

Outline

- Overview of Cryptography
- Classical Symmetric Cipher
 - Substitution Cipher
 - Transposition Cipher
- Modern Symmetric Ciphers (DES)

Symmetric Cipher Model



Requirements

- Two requirements for secure use of symmetric encryption:
 - a strong encryption algorithm
 - a secret key known only to sender / receiver
- $$Y = E_k(X)$$
- $$X = D_k(Y)$$
- Assume encryption algorithm is known
 - Implies a secure channel to distribute key

Classical Substitution Ciphers

- Letters of plaintext are replaced by other letters or by numbers or symbols
- Plaintext is viewed as a sequence of bits, then substitution replaces plaintext bit patterns with ciphertext bit patterns

Caesar Cipher

- Earliest known substitution cipher
- Replaces each letter by 3rd letter on
- Example:

meet me after the toga party

PHHW PH DIWHU WKH WRJD SDUWB

Caesar Cipher

- Define transformation as:

a b c d e f g h i j k l m n o p q r s t u v w x y z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

- Mathematically give each letter a number

a b c d e f g h i j k l m

0 1 2 3 4 5 6 7 8 9 10 11 12

n o p q r s t u v w x y z

13 14 15 16 17 18 19 20 21 22 23 24 25

- Then have Caesar cipher as:

$$C = E(p) = (p + k) \bmod (26)$$

$$p = D(C) = (C - k) \bmod (26)$$

Perform Encryption using
Mathematical calculation
"easy to break" with $k=2$

Cryptanalysis of Caesar Cipher

- Only have 25 possible ciphers
 - A maps to B,..Z
- Given ciphertext, just try all shifts of letters
- Do need to recognize when have plaintext
- E.g., break ciphertext "GCUA VQ DTGCM"

Monoalphabetic Cipher

- Rather than just shifting the alphabet
- Could shuffle (jumble) the letters arbitrarily
- Each plaintext letter maps to a different random ciphertext letter
- Key is 26 letters long

Plain: abcdefghijklmnopqrstuvwxyz

Cipher: DKVQFIBJWPESCXHTMYAUOLRGZN
 [based on some frequencies of letters]

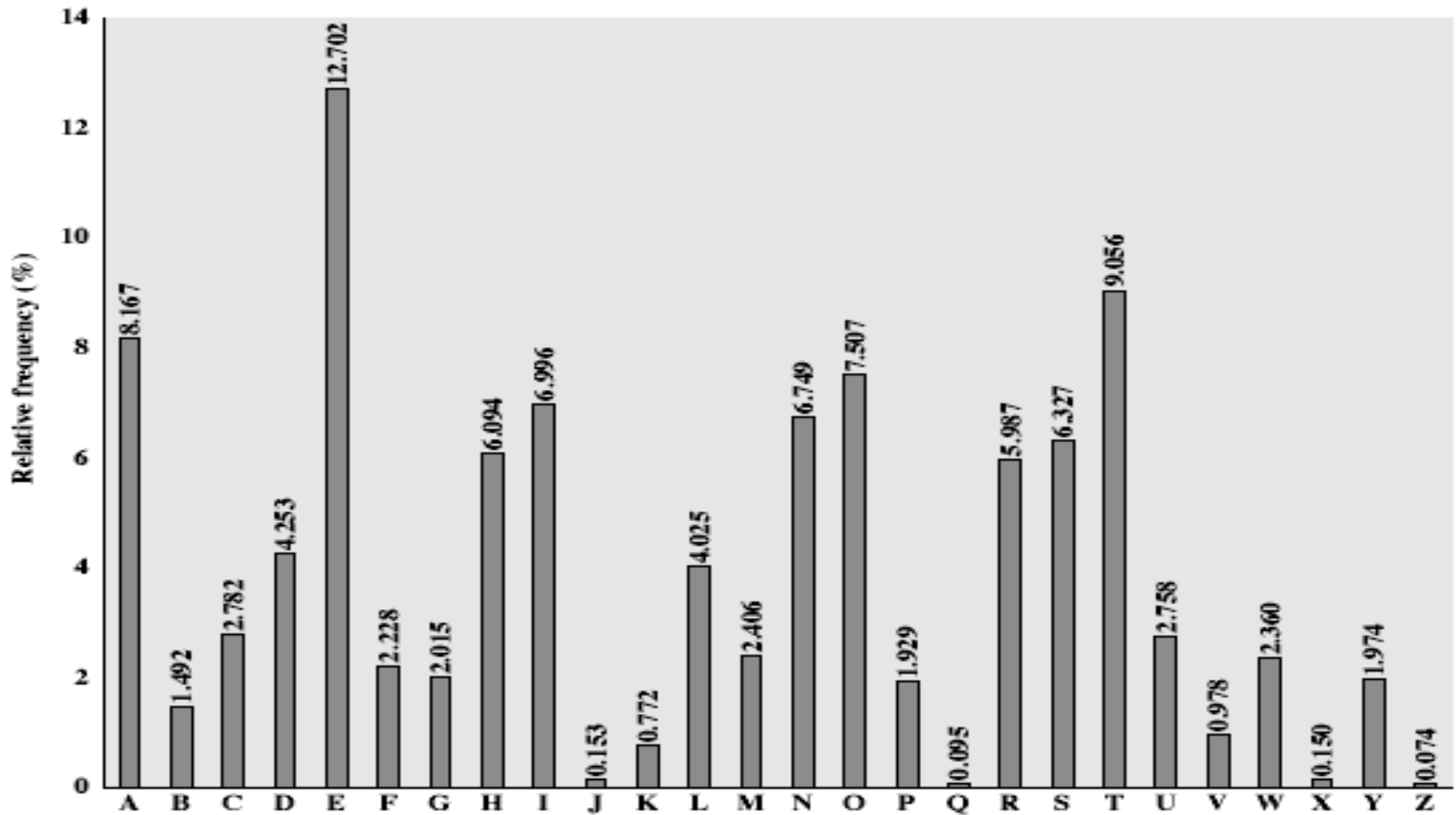
Plaintext: ifwewishtoreplaceletters

Ciphertext: WIRFRWAJUHYFTSDVFSFUUFYA

Monoalphabetic Cipher Security

- Now have a total of 26 permutation
 - ($26! = 4 \times 10^{26}$ keys)
- Is that secure?
- Problem is language characteristics
 - Human languages are **redundant**
 - Letters are not equally commonly used

English Letter Frequencies



Note that all human languages have varying letter frequencies, though the number of letters and their frequencies varies.

Example Cryptanalysis

Given ciphertext:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETS

VUEPHZHMDZSHZOWSFPAPDTSVPQUZWYMXUZUHSX

EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

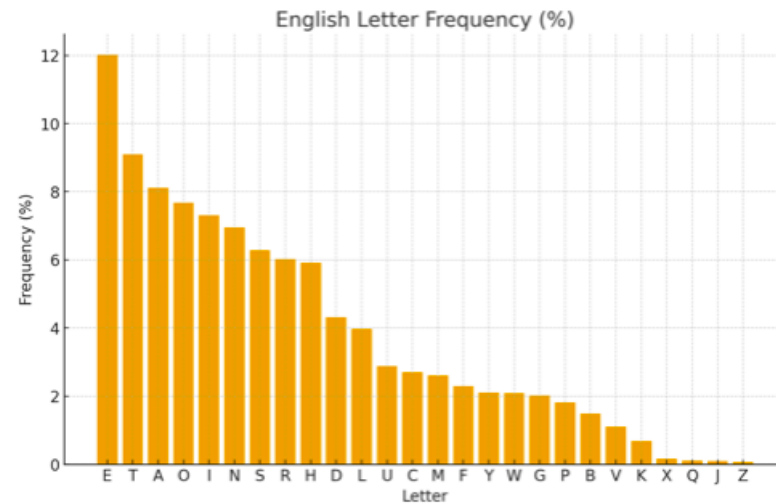
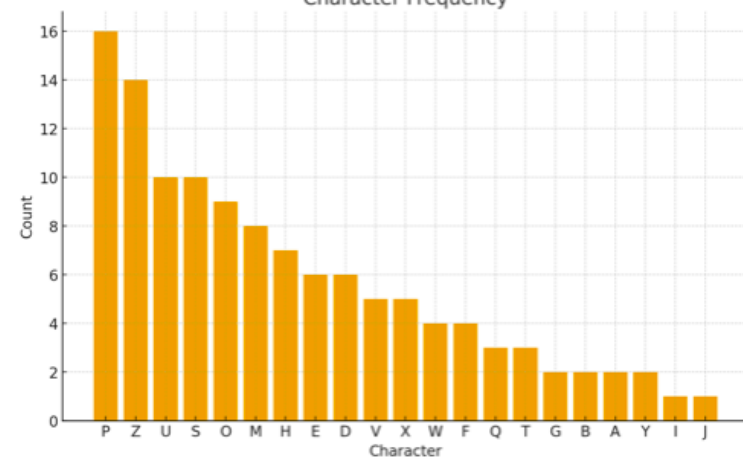
Count relative letter frequencies (s

Guess P & Z are e and t

Guess ZW=th & ZWP=the

Proceeding with trial and error finally get:

it was disclosed yesterday that several informal but
direct contacts have been made with political
representatives of the viet cong in moscow



One-Time Pad

- If a truly random key as long as the message is used, the cipher will be secure - One-Time pad
- E.g., a random sequence of 0's and 1's XORed to plaintext, no repetition of keys
- Unbreakable since ciphertext bears no statistical relationship to the plaintext
- For **any** plaintext, it needs a random key of the same length
 - Hard to generate large amount of keys
- Have problem of safe distribution of key

Example

Example (Binary Form):

Plaintext	1	0	1	1
Random Key	0	1	0	1
Ciphertext	$1 \oplus 0 = 1$	$0 \oplus 1 = 1$	$1 \oplus 0 = 1$	$1 \oplus 1 = 0$

Ciphertext = 1110

To decrypt: Ciphertext $\oplus$ Key $\rightarrow$ Original Plaintext.

Transposition Ciphers

- Now consider classical **transposition** or **permutation** ciphers
- These hide the message by rearranging the letter order, without altering the actual letters used
- Can recognise these since have the same frequency distribution as the original text

Encryption Process — Columnar Transposition Cipher

Plaintext: HELLOWORLD (remove 'space' for simplicity)

Key: 3142

Write the message in 4 columns according to the key:

H E L L

O W O R

L D X X

Now, read the columns in the order given by the key ($3 \rightarrow 1 \rightarrow 4 \rightarrow 2$):

Column 3: L O X

Column 1: H O L

Column 4: L R X

Column 2: E W D

Combine them to get the ciphertext:

Ciphertext: LOXHOLLRXEWD

Decryption

Ciphertext: LOXHOLLRXEWD

according to key order (3 → 1 → 4 → 2), 3 letters per column:

Key digit	Column data	Goes to original position
3	LOX	→ position 1
1	HOL	→ position 2
4	LRX	→ position 3
2	EWD	→ position 4

Re-order columns back to their **original positions (1 2 3 4)**:

pos1	pos2	pos3	pos4
H O L	E W D	L O X	L R X

Row 1: H E L L

Row 2: O W O R

Row 3: L D X X

Combine: **HELLOWORLDXX** → remove padding **XX**

Recovered Plaintext: HELLOWORLD

Rail Fence Cipher

- Write message letters out diagonally over a number of rows
- Then read off cipher row by row
- E.g., write message out as:

m e m a t r h t g p r y

e t e f e t e o a a t

- Giving ciphertext

MEMATRHTGPRYETEFETEOAAT

Rail Fence Cipher

Encryption:

Plaintext: HELLOWORLD

Length: 10

Rails: 2

Zigzag with 2 rails (indices 0..9):

Rail 0: H . L . O . O . L .

0 2 4 6 8

Rail 1: . E . L . W . R . D

1 3 5 7 9

Read Rail 0 left→right,
then Rail 1 left→right:

Rail 0 = H L O O L → HLOOL

Rail 1 = E L W R D → ELWRD

Ciphertext = HLOOL + ELWRD
= HLOOLELWRD

Decryption:

Ciphertext: HLOOLELWRD

Compute chars per rail.

For 2 rails, length 10:

Rail 0, length = $\text{ceil}(10/2) = 5$

Rail 1, length = $\text{floor}(10/2) = 5$

Split ciphertext into those lengths:

Rail 0 ← first 5 chars → H L O O L

Rail 1 ← next 5 chars → E L W R D

Reconstruct by zigzag (alternating rails
for indices 0..9):

index: 0 1 2 3 4 5 6 7 8 9

rail : 0 1 0 1 0 1 0 1 0 1

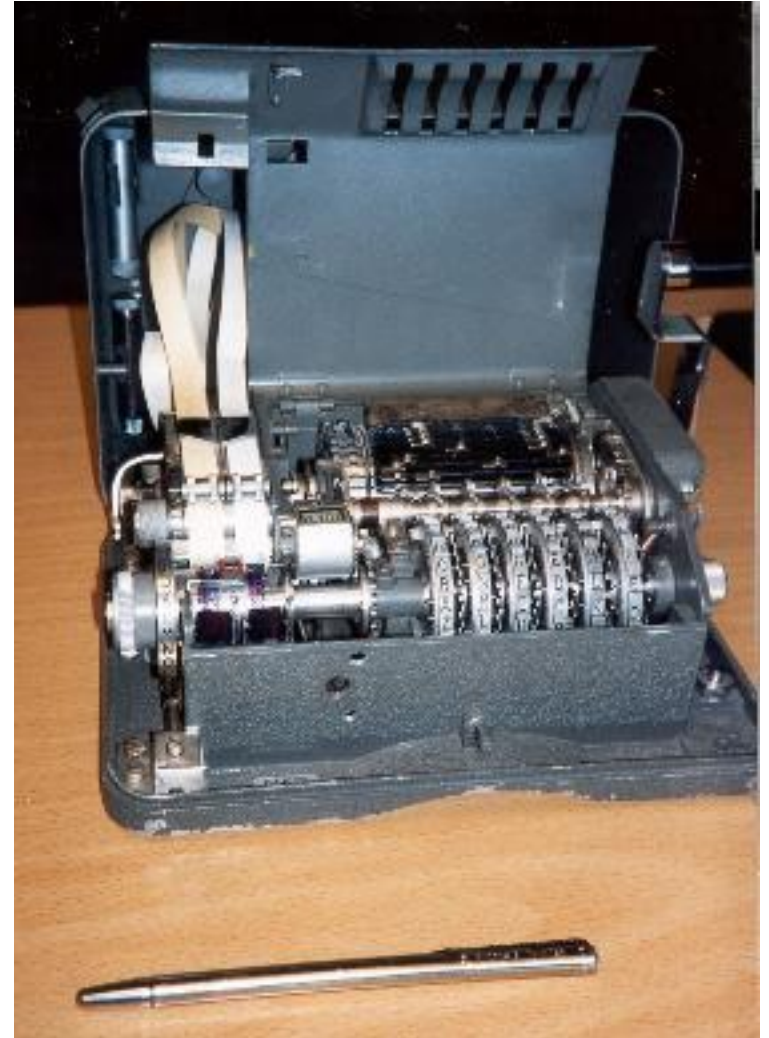
plaintext: H E L L O W O R L D

Product Ciphers

- Ciphers using substitutions or transpositions are not secure because of language characteristics
- Hence consider using several ciphers in succession to make harder, but:
 - Two substitutions make another substitution
 - Two transpositions make a more complex transposition
 - But a substitution followed by a transposition makes a new much harder cipher
- This is bridge from classical to modern ciphers

Rotor Machines

- Before modern ciphers, rotor machines were most common complex ciphers in use
- Widely used in WW2
 - German Enigma, Allied Hagelin, Japanese Purple
- Implemented a very complex, varying substitution cipher



Outline

- Overview of Cryptography
- Classical Symmetric Cipher
- Modern Symmetric Ciphers (DES)

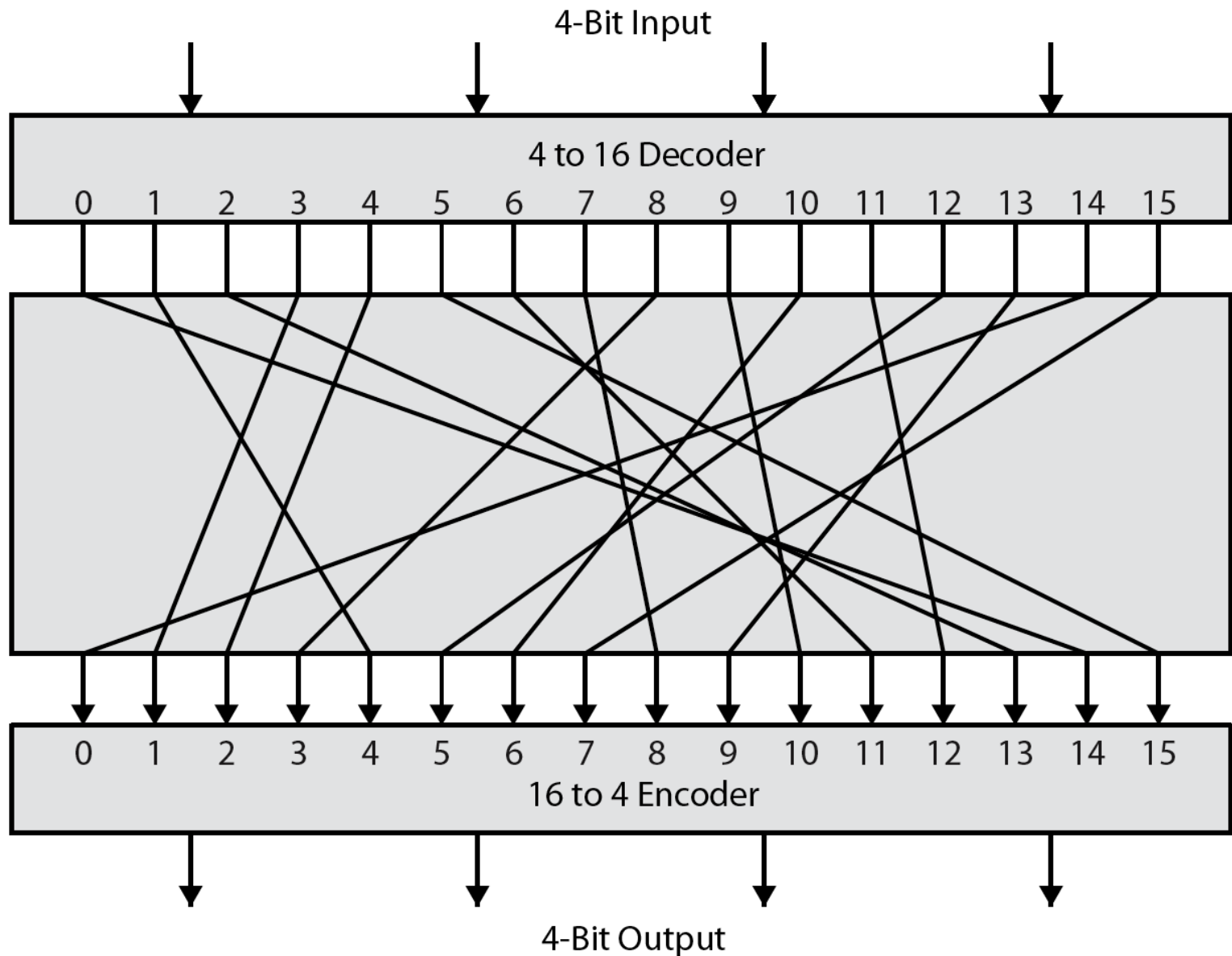
Block vs Stream Ciphers

- Block ciphers process messages in into blocks, each of which is then en/decrypted
 - Like a substitution on very big characters
 - 64-bits or more
- Stream ciphers process messages a bit or byte at a time when en/decrypting
 - Many current ciphers are block ciphers, one of the most widely used types of cryptographic algorithms

Block Cipher Principles

- Most symmetric block ciphers are based on a **Feistel Cipher Structure**
- Block ciphers look like an extremely large substitution
- Would need table of 2^{64} entries for a 64-bit block
- Instead create from smaller building blocks
- Using idea of a product cipher

Ideal Block Cipher



Substitution-Permutation Ciphers

- Substitution-permutation (S-P) networks
[Shannon, 1949]
 - modern substitution-transposition product cipher
- These form the basis of modern block ciphers
- S-P networks are based on the two primitive cryptographic operations
 - *substitution* (S-box)
 - *permutation* (P-box)
- provide *confusion* and *diffusion* of message

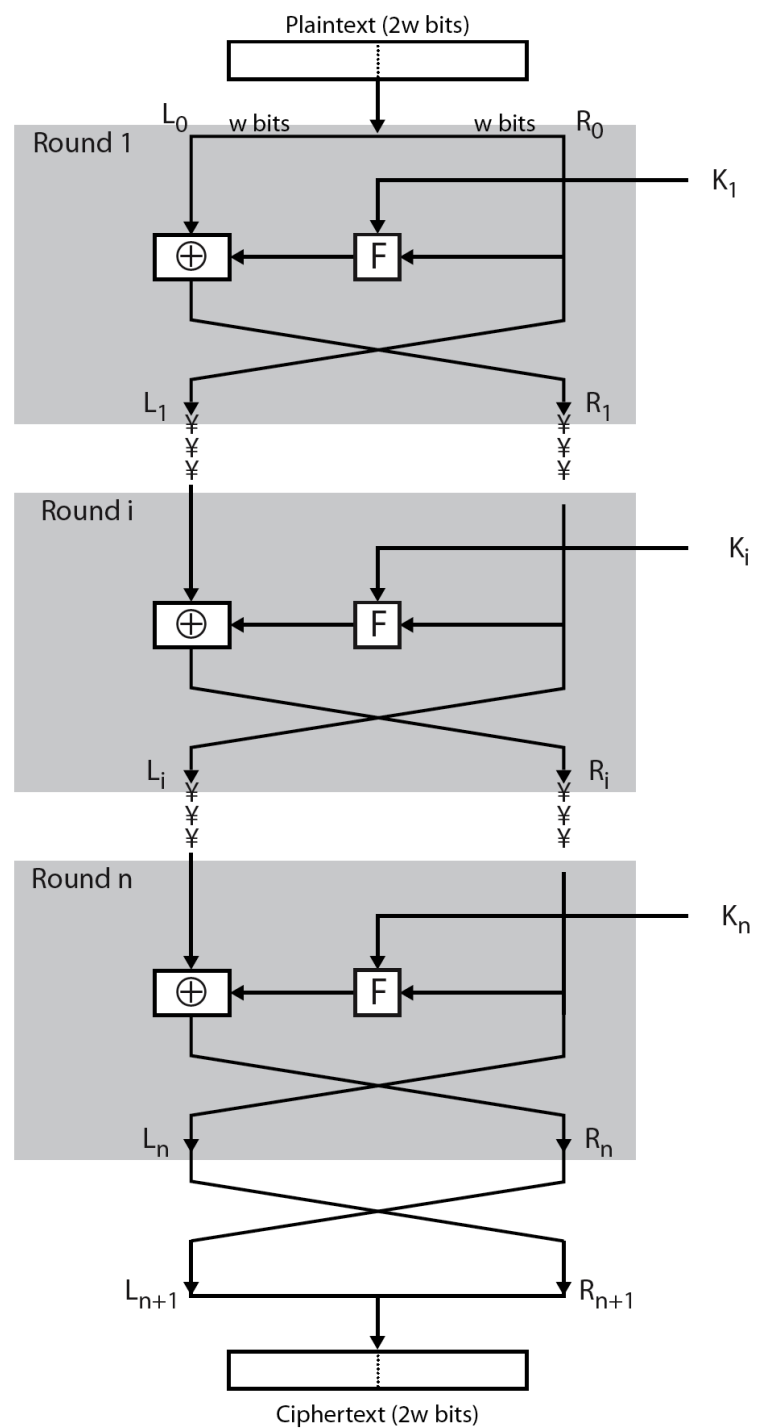
Confusion and Diffusion

- Cipher needs to completely obscure statistical properties of original message
- A one-time pad does this
- More practically Shannon suggested S-P networks to obtain:
- **Diffusion** - dissipates statistical structure of plaintext over bulk of ciphertext
- **Confusion** - makes relationship between ciphertext and key as complex as possible

Feistel Cipher Structure

- **Feistel cipher** implements Shannon's S-P network concept
 - based on invertible product cipher
- Process through multiple rounds which
 - partitions input block into two halves
 - perform a substitution on left data half
 - based on round function of right half & subkey
 - then have permutation swapping halves

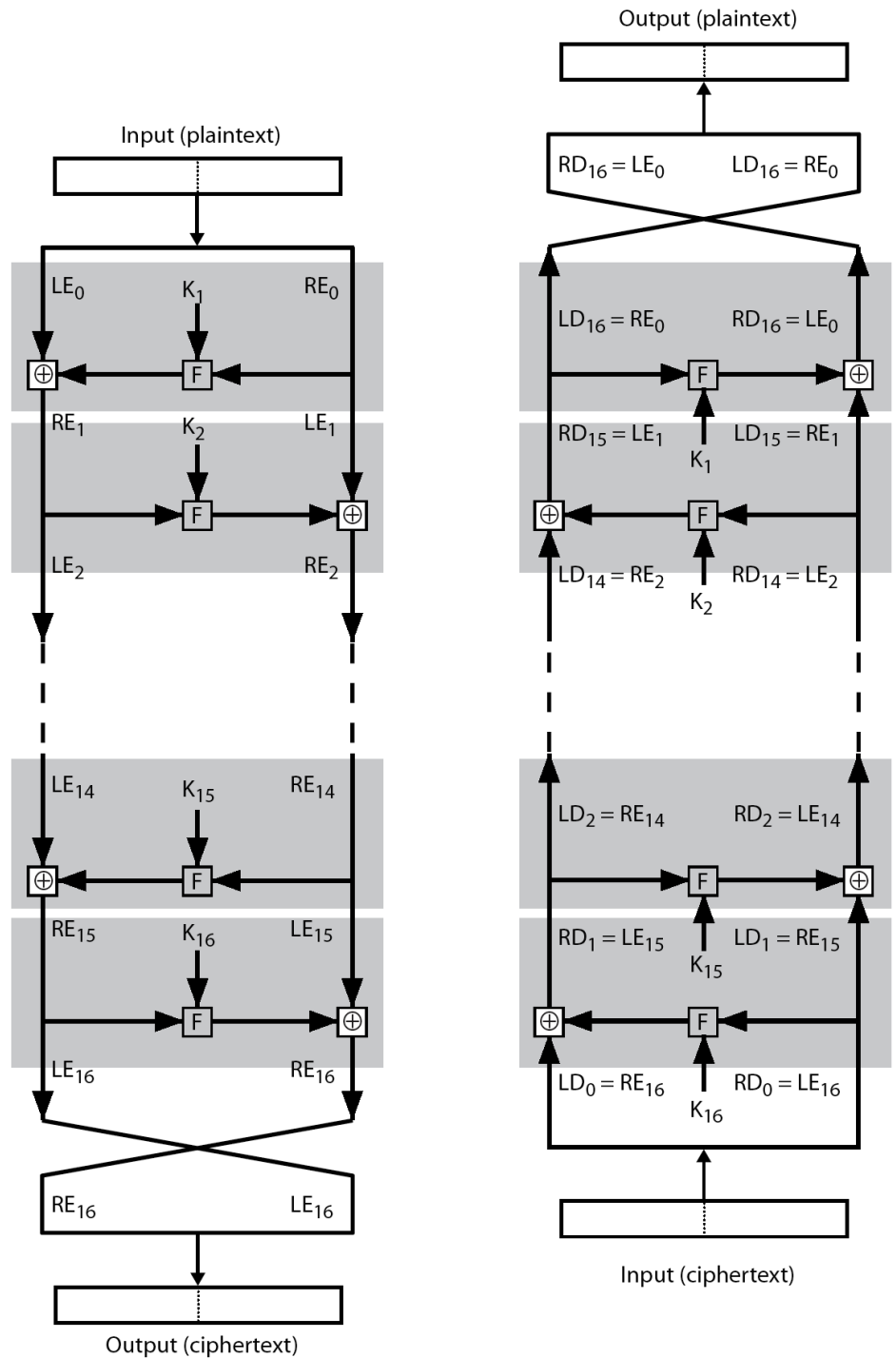
Feistel Cipher Structure



Feistel Cipher Structure (Encryption)

- **Given:**
 - Block size 8 bits
 - No. of rounds 2
 - Round key K_1 1010
 - Round key K_2 0101
- Plaintext 11010111
- **Step 1 — Initial Split**
 - Split the 8-bit plaintext into two 4-bit halves:
 - $L_0 = 1101$, $R_0 = 0111$
- **Step 2 — Round 1**
 - Feistel uses:
 - $L_1 = R_0$, $R_1 = L_0 \oplus F(R_0, K_1)$, where $F(R, K) = R \oplus K$
 - Compute:
 - $F(R_0, K_1) = 0111 \oplus 1010 = 1101$
 - $R_1 = 1101 \oplus 1101 = 0000$
 - $L_1 = R_0 = 0111$
 - After Round 1:
 - $L_1 = 0111$, $R_1 = 0000$
- **Step 3 — Round 2**
 - Feistel uses:
 - $L_2 = R_1$, $R_2 = L_1 \oplus F(R_1, K_2)$
 - Compute:
 - $F(R_1, K_2) = 0000 \oplus 0101 = 0101$
 - $R_2 = 0111 \oplus 0101 = 0010$
 - $L_2 = 0000$
 - After Round 2:
 - $L_2 = 0000$, $R_2 = 0010$
- **Step 4 — Combine**
 - Final ciphertext = $L_2 \parallel R_2 =$
 - $0000 \ 0010 \rightarrow 00000010$

Feistel Cipher Decryption



Feistel Cipher Structure (Decryption)

- Given:

- Ciphertext 00000010
- Block size 8 bits
- No. of rounds 2
- Keys (reversed) $K_2 = 0101$, $K_1 = 1010$

- Step 1 — Split Ciphertext

- $L_2 = 0000$, $R_2 = 0010$

- Step 2 — Round 2
(reverse of encryption Round 2)

- Use the Feistel equations in reverse:
- $R_1 = L_2$, $L_1 = R_2 \oplus F(R_1, K_2)$
- Compute:
- $R_1 = L_2 = 0000$
- $F(R_1, K_2) = 0000 \oplus 0101 = 0101$
- $L_1 = R_2 \oplus F(R_1, K_2) = 0010 \oplus 0101 = 0111$
- After Round 2:
- $L_1 = 0111$, $R_1 = 0000$

- Step 3 — Round 1
(reverse of encryption Round 1)

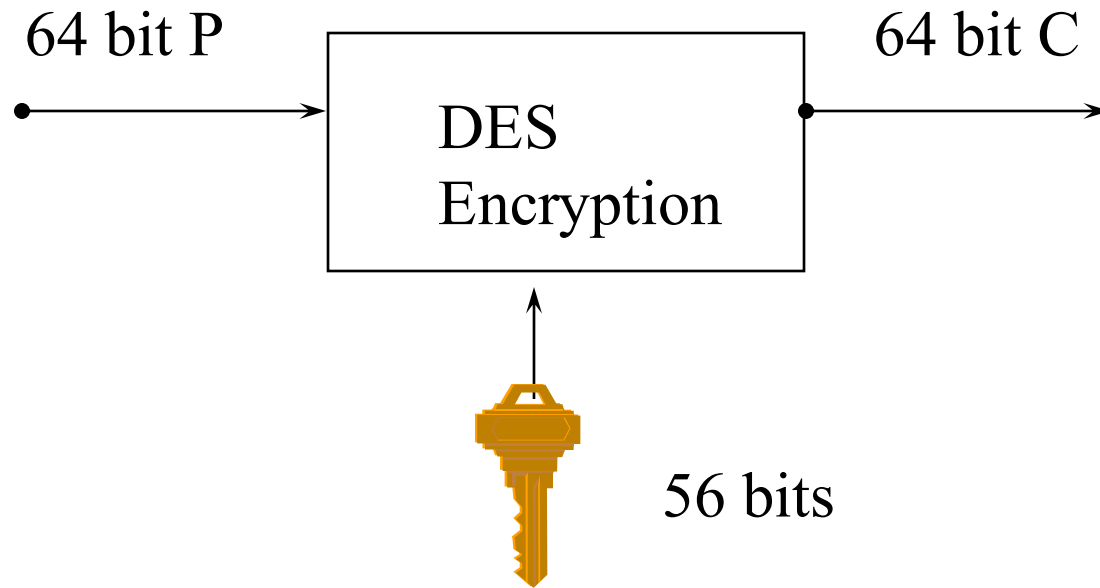
- Use the Feistel equations in reverse :
- $R_0 = L_1$, $L_0 = R_1 \oplus F(R_0, K_1)$
- Compute:
- $R_0 = L_1 = 0111$
- $F(R_0, K_1) = 0111 \oplus 1010 = 1101$
- $L_0 = R_1 \oplus F(R_0, K_1) = 0000 \oplus 1101 = 1101$
- After Round 1:
- $L_0 = 1101$, $R_0 = 0111$

- Step 4 — Combine (Rebuild Plaintext)

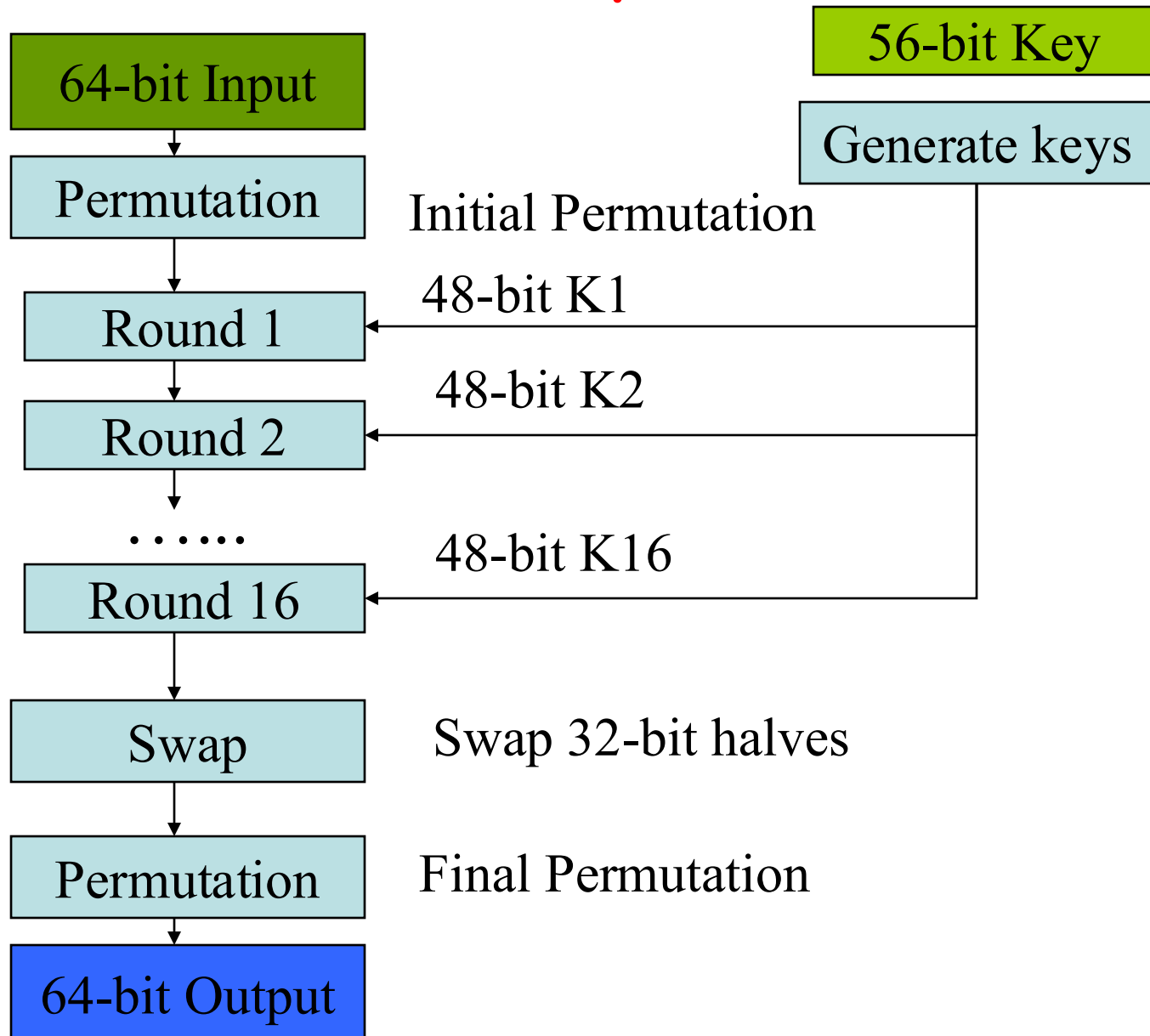
- Plaintext = $L_0 \parallel R_0 = 1101 \parallel 0111 = 11010111$

DES (Data Encryption Standard)

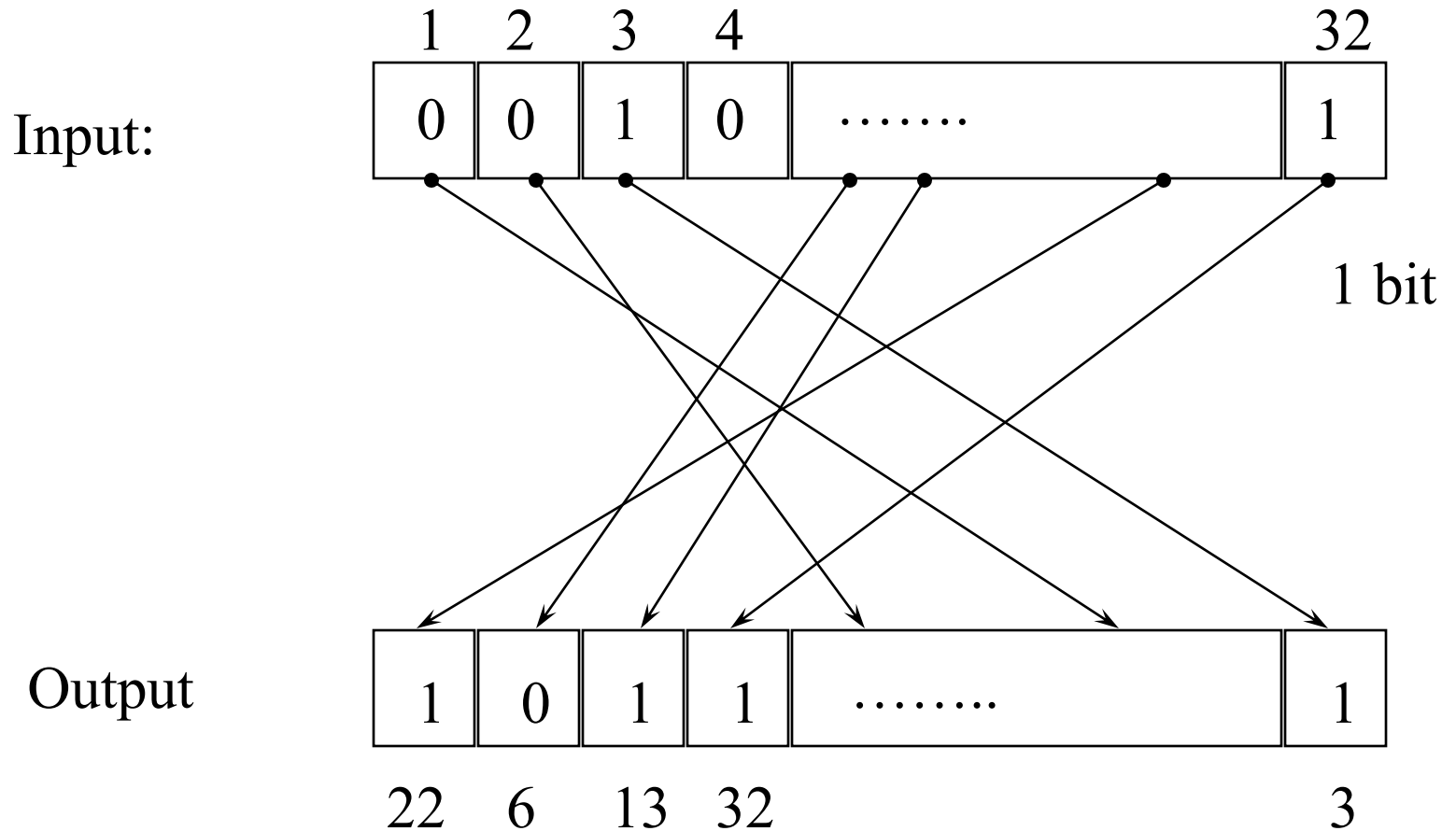
- Published in 1977, standardized in 1979.
- Key: 64 bit, quantity=8-bit parity + 56-bit key
 - Every 8th bit is a parity bit.
- 64 bit input, 64 bit output.



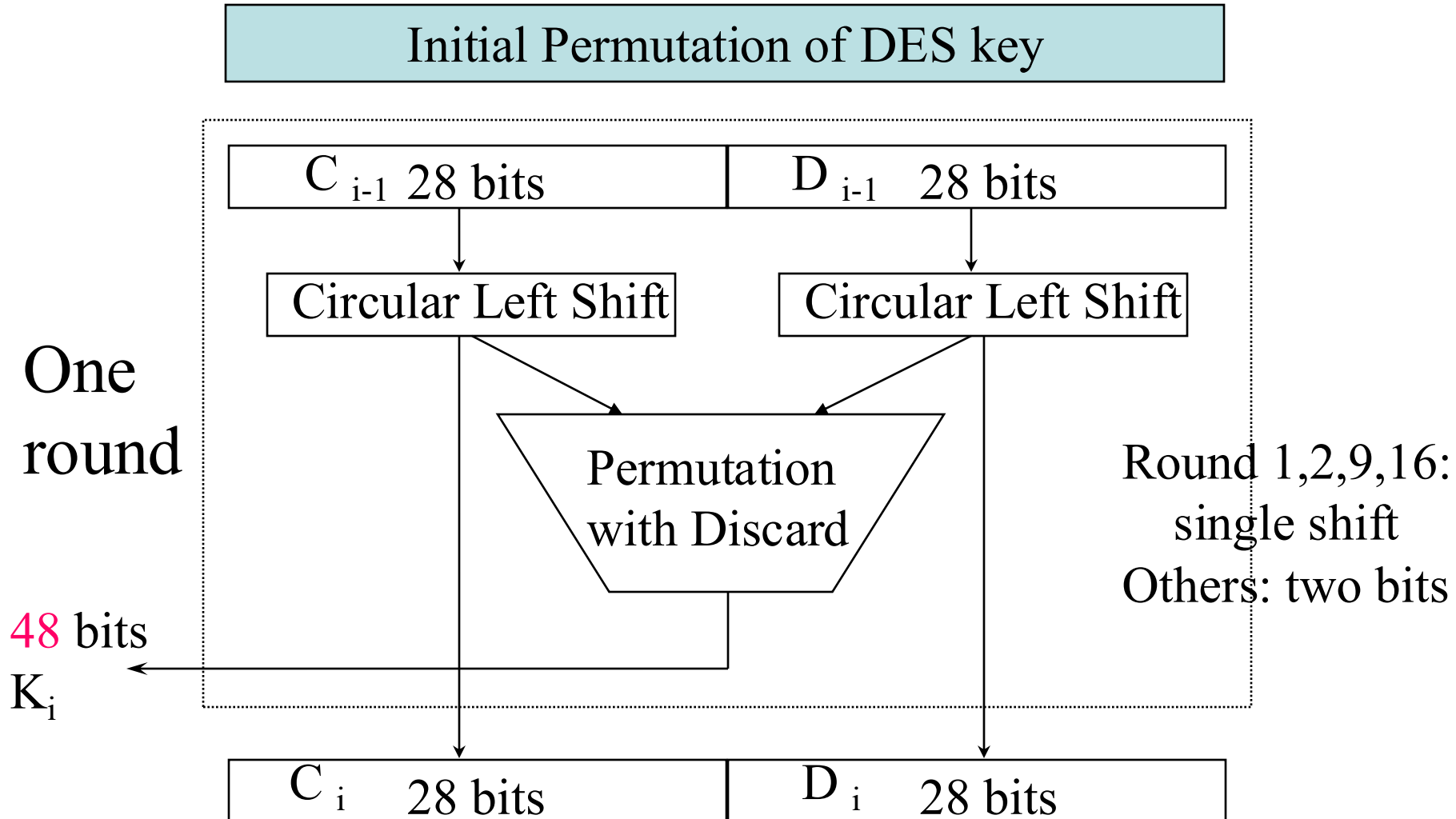
DES Top View



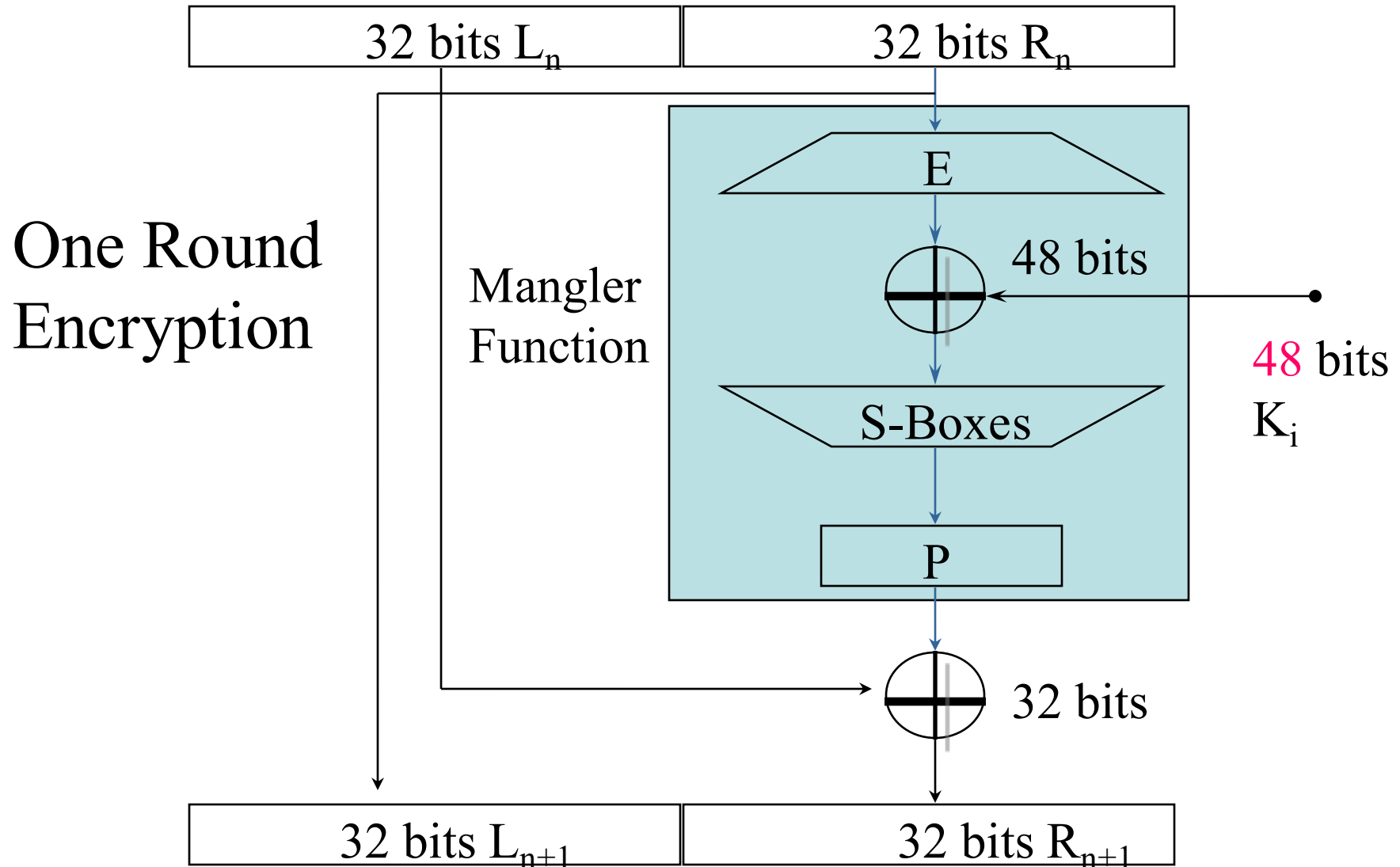
Bit Permutation (1-to-1)



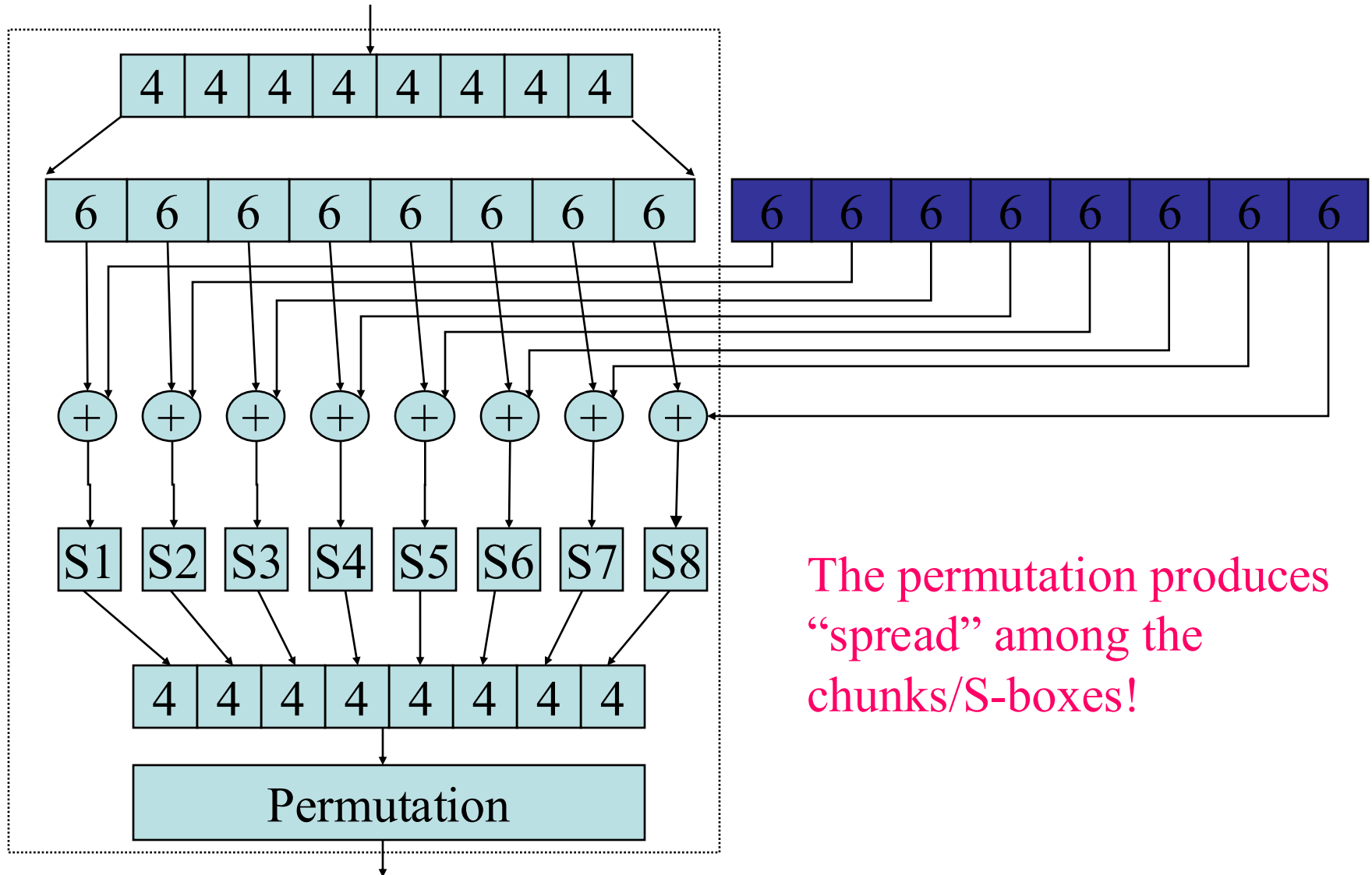
Per-Round Key Generation



A DES Round

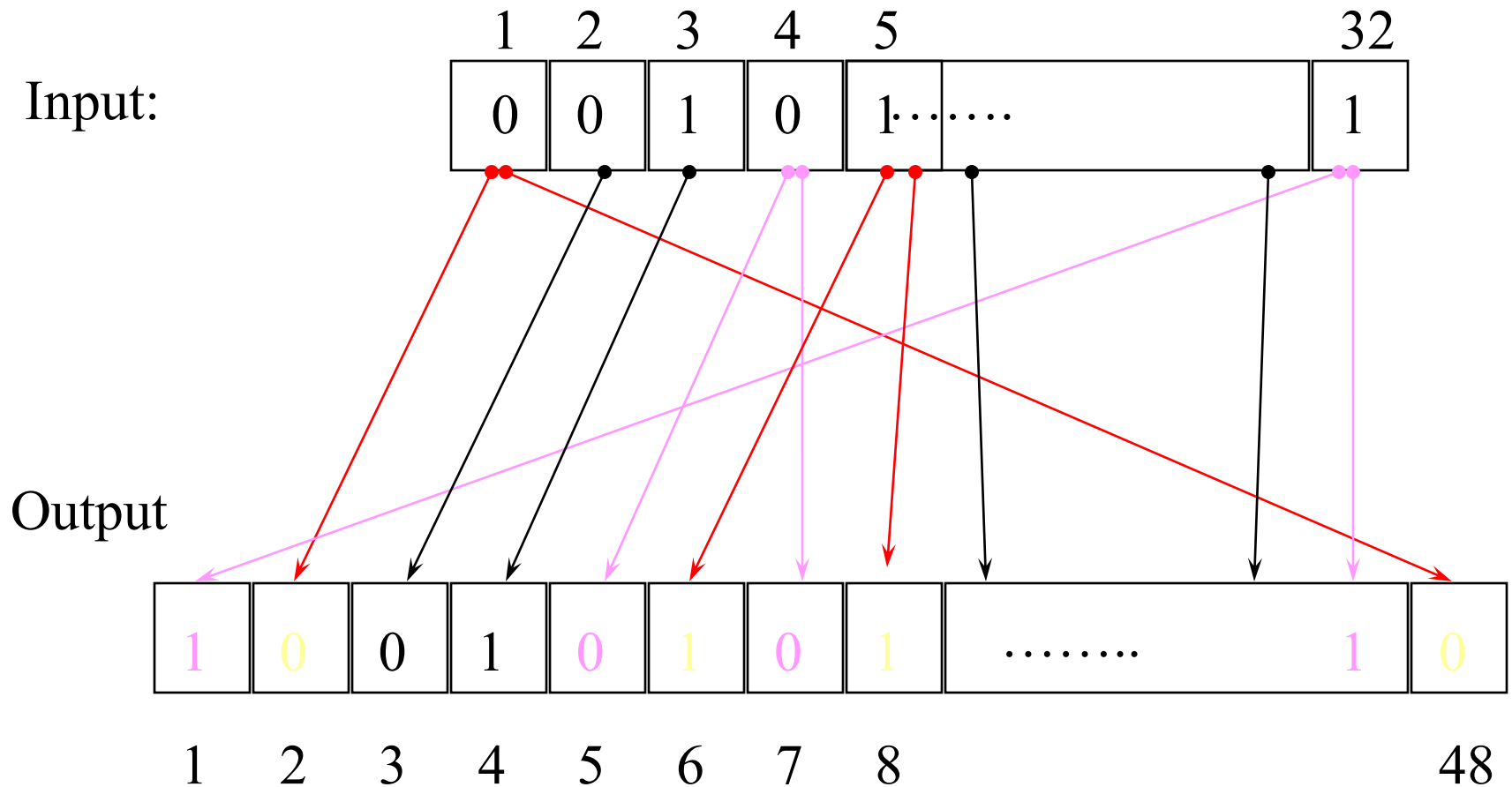


Mangler Function



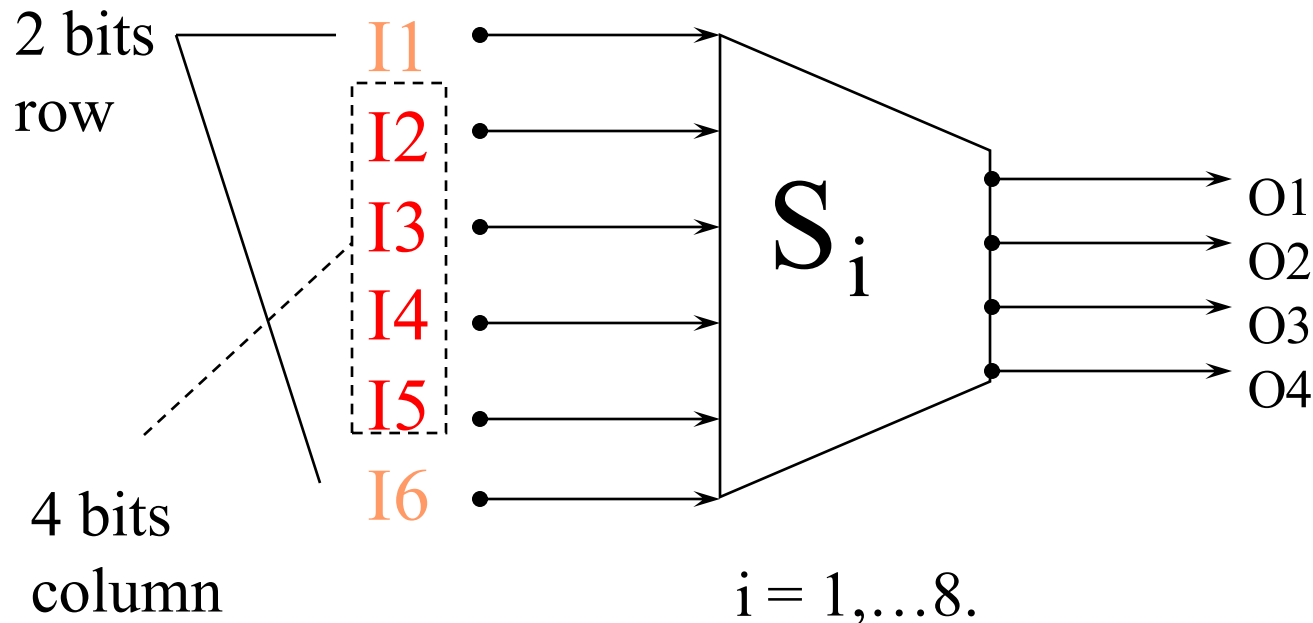
The permutation produces
“spread” among the
chunks/S-boxes!

Bits Expansion (1-to-m)



S-Box (Substitute and Shrink)

- 48 bits $\Rightarrow$ 32 bits. ($8 \times 6 \Rightarrow 8 \times 4$)
- 2 bits used to select amongst 4 substitutions for the rest of the 4-bit quantity



S-Box Examples

Each row and column contain different numbers.

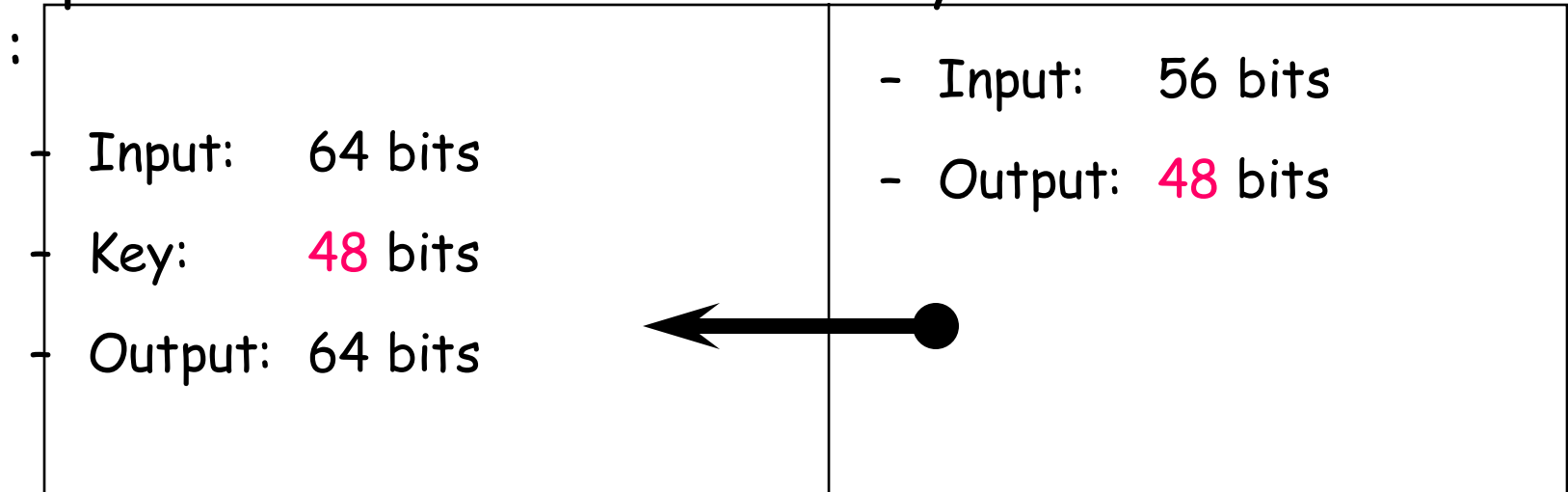
	0	1	2	3	4	5	6	7	8	9.... 15
0	14	4	13	1	2	15	11	8	3	
1	0	15	7	4	14	2	13	1	10	
2	4	1	14	8	13	6	2	11	15	
3	15	12	8	2	4	9	1	7	5	

Example: input: 100110 output: ???

DES Standard

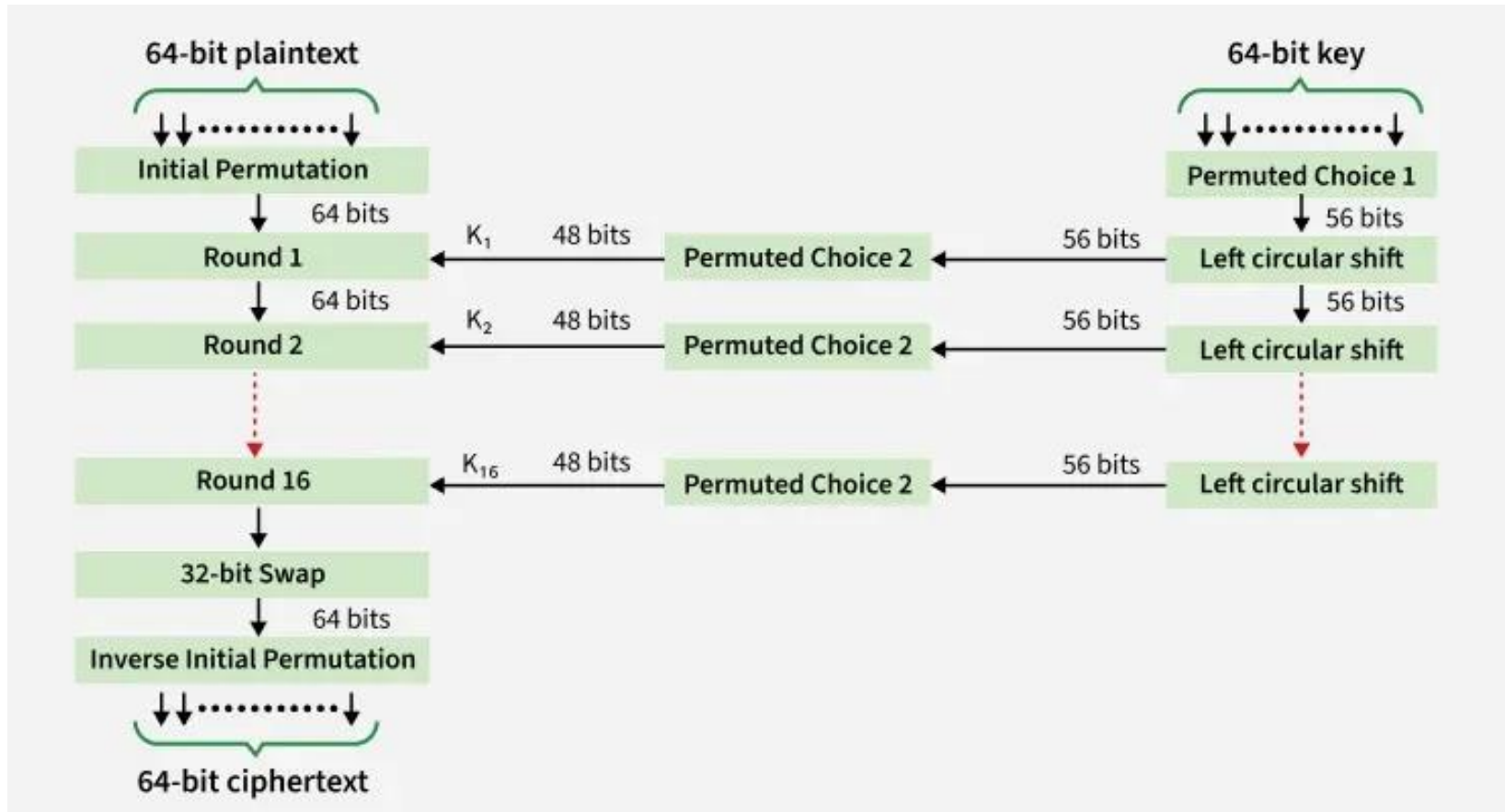
- Cipher Iterative Action

- Key Generation Box :



One round (Total 16 rounds)

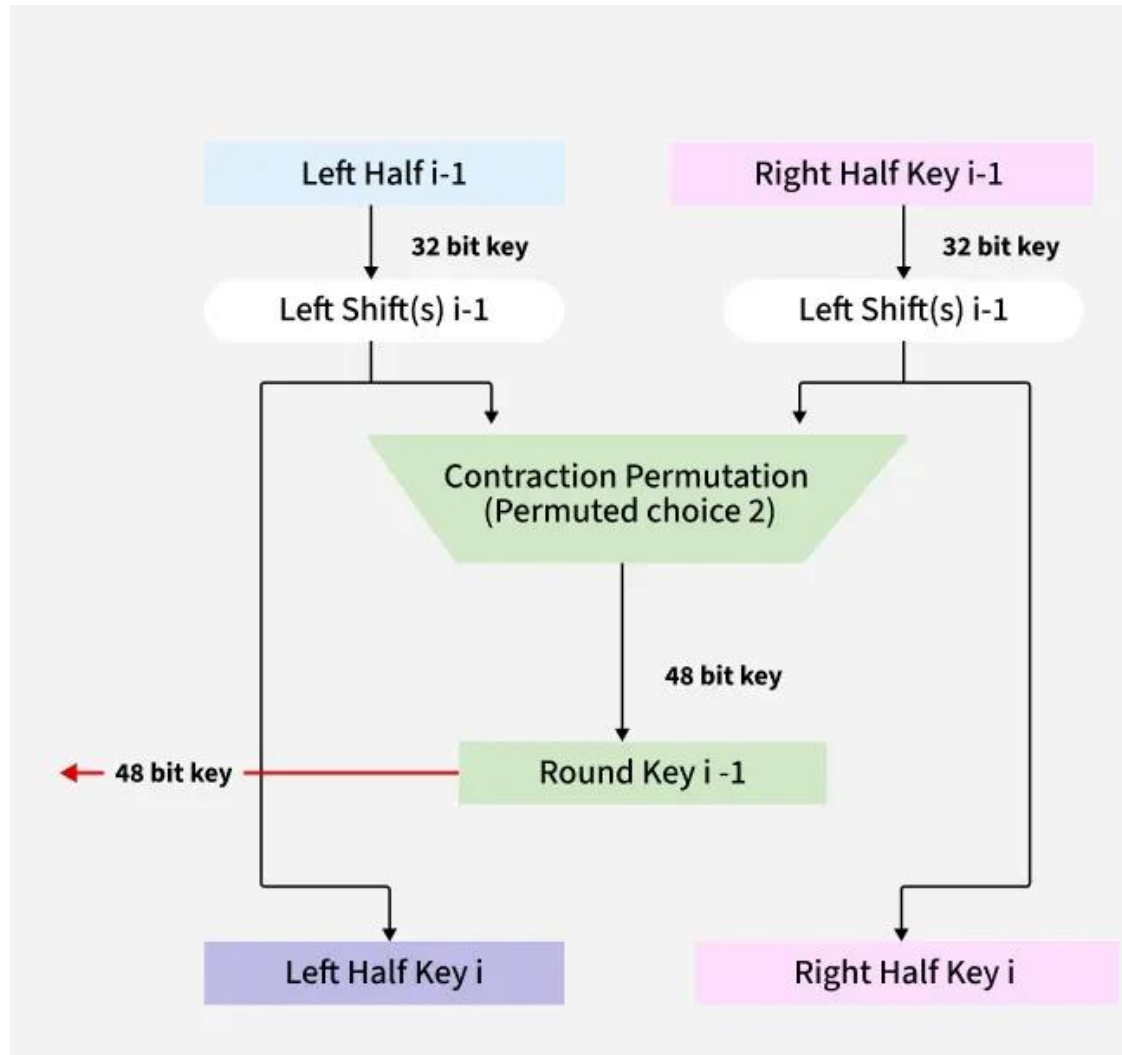
DES



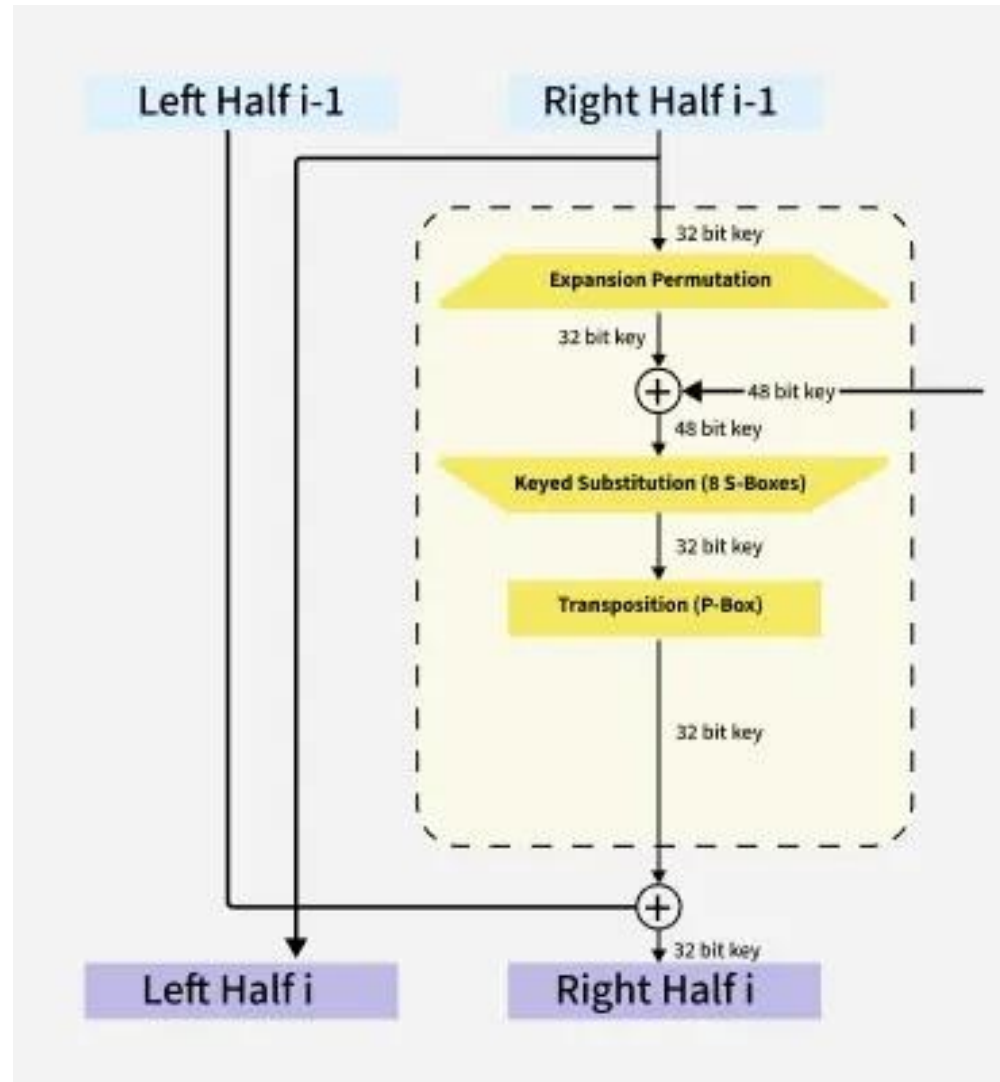
Plaintext: 64 bits

Key: 64 bits

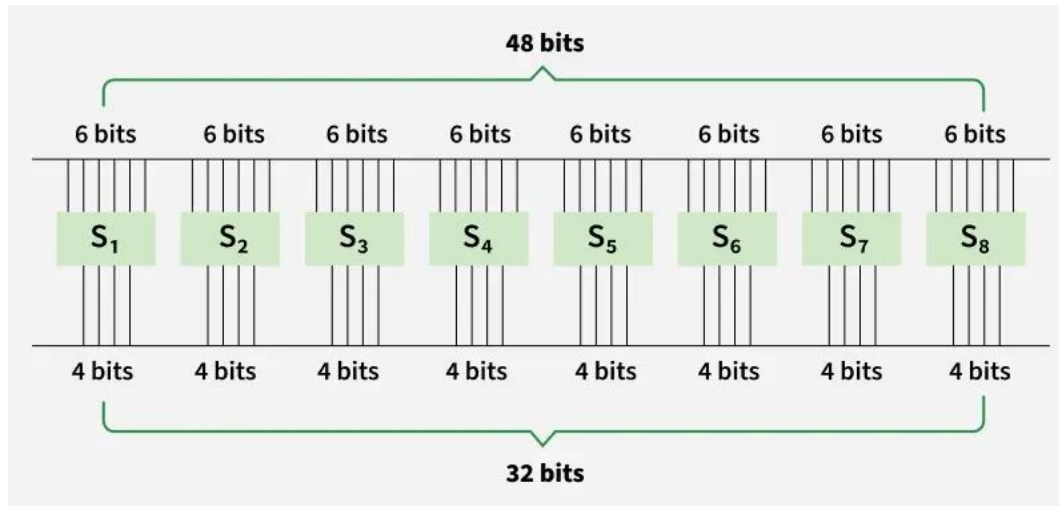
Key Generation



Single Feistel Round in DES



S-Box

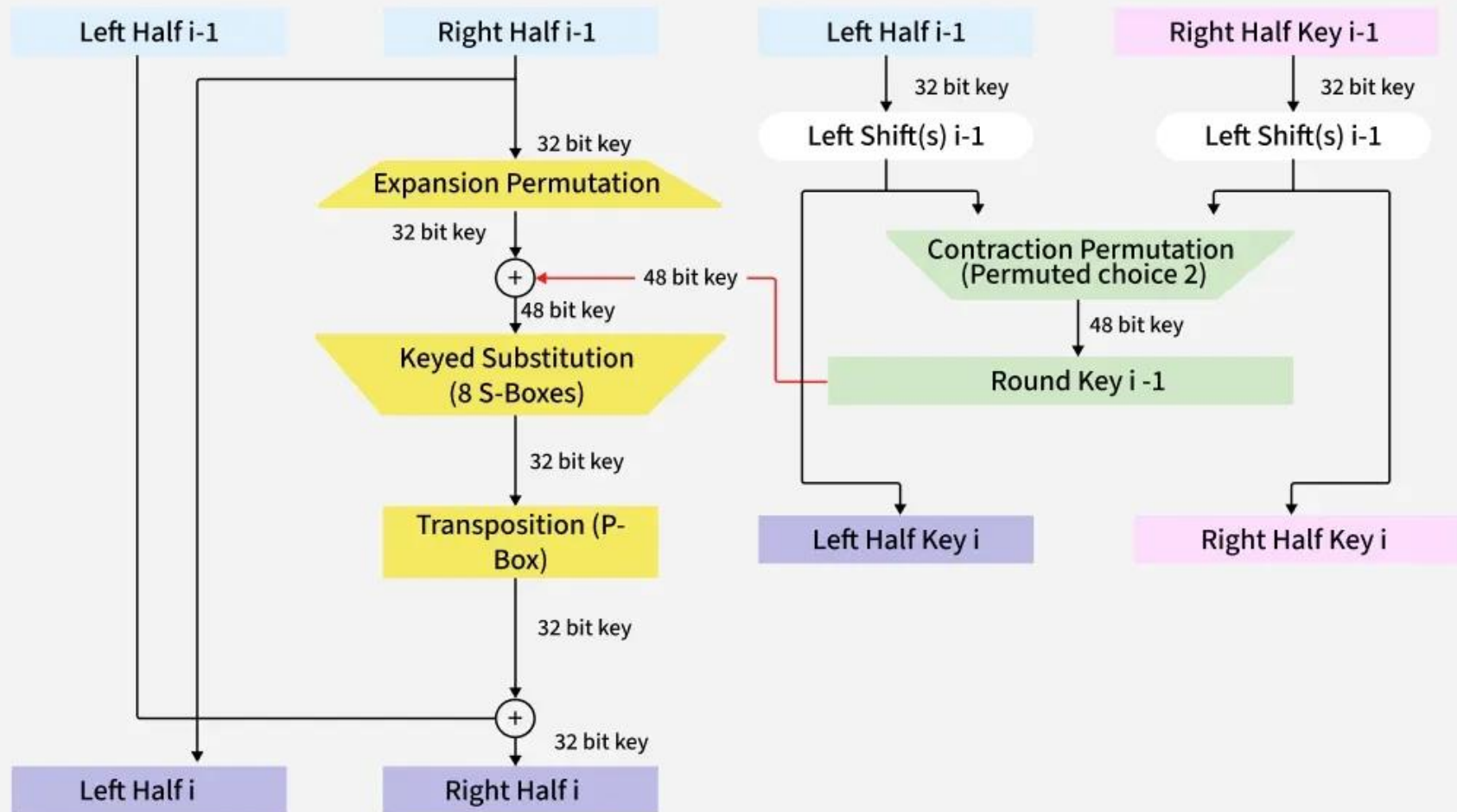


6-bit chunk: '101010' converted into 4-bit chunk: '0110'

SBox-1

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
00	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
01	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
10	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
11	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Mangler Function & Key Transformation



DES Box Summary

- Simple, easy to implement:
 - Hardware/gigabits/second,
software/megabits/second
- 56-bit key DES may be acceptable for non-critical applications but triple DES (DES3) should be secure for most applications today
- Supports several operation modes (ECB CBC, OFB, CFB) for different applications

- Is DES still used today?
- DES - Retired in 2005 (56 bits key)
- 3DES - Retired in 2023 (112 bits key)
- AES - in service (128-256 bits key)